Meeting Scribe: Technical Report

Krishang Vyas · Inter IIT Tech Meet 15.0 (IIT Guwahati Bootcamp), ML problem statement

1. What it does

Meeting Scribe takes an English meeting recording and, in one run, produces a raw transcript, a refined transcript, and a meeting record (summary, minutes by topic, decisions, action items, open questions) in both a readable format (Markdown, HTML, PDF) and a structured one (JSON).

The design rule is **the AI suggests, plain code checks**. Each model's output goes through deterministic checks before the next stage uses it. Anything that fails a check is shown under *Needs review* with its timestamp instead of being silently kept or dropped.

2. Models and their roles

Stage	Model	Role
Speech-to-text	**Whisper Large V3 Turbo** (`whisper-large-v3-turbo`), served by Groq	Turns audio into text with start/end times for every segment and word, plus per-segment confidence (`avg_logprob`, `no_speech_prob`, `compression_ratio`).
Language model 1: **refiner**	**GPT-OSS 120B** (`openai/gpt-oss-120b`), served by Groq	Fixes speech-recognition errors: technical terms, names, acronyms, sound-alike words, words wrongly split or joined. It only proposes small edits; it never rewrites the transcript.
Language model 2: **documenter**	**GPT-OSS 120B** (`openai/gpt-oss-120b`), served by Groq	Reads the refined transcript and writes topics with minutes, decisions (agreed / proposed / open), action items (owner, deadline, commitment level), and a summary. Every item must quote the transcript.

The refiner and the documenter are separate stages: different prompts, different calls, different output files, run one after the other. Each model name is one line at the top of its file (`transcribe.py`, `refine.py`, `inventory.py`, `document.py`), so either stage can be switched to another model.

Why these models: Groq runs all three without a local GPU, has a free tier, and is fast. GPT-OSS 120B was the strongest model Groq offered without an enterprise plan, has a 131k-token context (a full hour of meeting fits), and supports JSON output.

3. How outputs move between stages

```
upload ──▶ 0 Validate ──▶ 1 Transcribe ──▶ 2 Refine ──▶ 3 Document ──▶ 4 Export
           audio_16k.flac  raw_transcript   edits.json    record.json     minutes.md/.html/.pdf
                           stt_log.json     refined_      record_draft    zip of the run
                                            transcript
```

Stages talk only through files saved in `runs/<run id>/`. Each stage reads what the stage before it saved, so a failed run can resume from the failed stage without transcribing again.

Stage 0: Validate (`validate.py`). ffprobe checks the upload has an audio stream and a sensible length; ffmpeg converts it to 16 kHz mono FLAC. Empty, damaged, silent, too short, or non-audio files (e.g. a text file renamed `.mp3`) stop here with a plain message.

Stage 1: Transcribe (`transcribe.py`, `chunks.py`, `shape.py`, `hallucination.py`). Groq accepts at most 25 MB per request, so long recordings are cut into ~10-minute pieces in the middle of silences, transcribed one by one, and their timestamps shifted back. Each segment gets an id and its own word list. Known Whisper hallucinations are removed and logged: repeated loops (compression ratio > 2.4), text in silence (high `no_speech_prob` with low `avg_logprob`), and stock phrases such as "thanks for watching". Output: `raw_transcript.json`.

Stage 2: Refine (`inventory.py`, `refine.py`, `guardrails.py`, `apply_edits.py`). First the refiner lists the meeting's domain terms and the forms it heard; code keeps a form only if it really occurs and sounds like the term, and keeps a person's name only if it was said. Then the refiner sees the transcript 20 segments at a time, with every word numbered, and proposes edits such as *"segment 4, words 4–7: 'cube and eighties' → 'Kubernetes'"*. Eight code checks decide each edit:

#	Check	Rule
1	Anchor	the words being replaced really are at those positions
2	Size	at most 4 words in and out
3	Numbers	numbers must stay the same; "fifteen" → "15" is not allowed either
4	Negation	not / never / n't can't be added or removed (*flagged for review*)
5	Commitment	will / should / must / going to can't change (*flagged for review*)
6	Sound-alike	new words must sound like the old ones (Metaphone similarity ≥ 0.5)
7	Supported	every new word was said in the meeting, is a checked term, or sounds identical (by → buy)
8	Overlap	two edits can't touch the same words

Once a misheard form is fixed, the same form elsewhere gets the same fix. Accepted edits are applied to a copy; the raw transcript is never changed. Outputs: `edits.json` (every edit with its status and the rule that decided it) and `refined_transcript.json`.

Stage 3: Document (`document.py`, `verify.py`). The documenter makes three calls on the refined transcript: topics with minutes points, then decisions/tasks/open questions, then the summary. Every item carries quotes with segment ids. Code then checks:

- each quote is really in the transcript (fuzzy match, then replaced by the exact words);
- an owner or deadline is kept only if it appears in a quote; otherwise it becomes *Unspecified*;
- a decision is *agreed* only if a quote shows acceptance ("yes", "agreed", "let's do it"); otherwise it is downgraded to *proposed*;
- a task is *confirmed* only with a "yes" or a volunteer ("I'll do it"); otherwise *assigned* (named person, no reply) or *suggested* (nobody named);
- items whose quotes can't be found get one repair attempt, then go to *Needs review*;
- duplicates are merged (never two items with different numbers);
- the summary is written last, only from checked items, and rejected if it contains a number or name nobody said; it is rewritten once, then flagged.

Outputs: `record_draft.json` (unchecked, kept for inspection) and `record.json`.

Stage 4: Export (`render.py`, `templates/minutes.md.j2`). A fixed template turns `record.json` into `minutes.md`, `minutes.html` and `minutes.pdf`, so the readable minutes and the JSON always contain the same decisions and tasks. Missing owners and deadlines are printed as *Unspecified*; deadlines are kept as spoken ("by Friday"), never turned into dates.

Reproducibility. Temperature is 0 and every LLM answer is cached under a hash of the model, prompt and input, so the same recording gives the same output.

4. Interface

`app.py` (Gradio) lets the user upload audio or video, shows each stage as it runs, and has tabs for the meeting record, raw transcript, refined transcript (accepted changes highlighted, refused edits shown with their rule), the edits log, *Needs review*, and downloads (every file plus a zip). Every timestamp is clickable and jumps the audio player to that moment. If a stage after validation fails, a *Retry* button resumes from the saved files.

5. Results on the sample meeting

The sample (`sample/`) is a 74-second hackathon planning meeting recorded on a laptop, written to contain the cases the problem statement warns about.

Check	Expected	Output
"React and Vercel" → "yes, agreed"	agreed	agreed
"Let's not [add login], there isn't enough time"	agreed (not doing it)	agreed
"What if we use Firebase?" → "hmm, maybe"	proposed only	proposed
"Cash or vouchers? Let's decide next meeting"	open question	open question
"Rahul, can you … by Friday?" → "Yes, I'll do that"	Rahul, by Friday, confirmed	same
"Sneha, please make the poster by Wednesday" (no reply)	Sneha, by Wednesday, assigned	same
"Someone should check the WiFi"	owner and deadline Unspecified, suggested	same
"fifty seats, sorry, sixty seats"	60	summary says 60

All 3 decisions, 1 open question and 3 action items match; no owner or deadline was invented. The refiner fixed the mishearing "Hekthon" → "Hackathon". On a separate check with planted mishearings (`eval/refine_check.py`) it fixed all six (cube and eighties → Kubernetes, get hub → GitHub, by → buy, a p i → API, post grass → PostgreSQL) and left "fifteen" and "can't" untouched.

`eval/` also holds four scripted trap meetings with written answers; `eval/score.py` runs the pipeline on them and reports term recall, WER, decision and task accuracy, and the number of invented owners or deadlines.

6. Limitations

- **No speaker labels.** Groq's Whisper does not identify speakers, so owners come only from names said in the quotes. A task someone volunteered for without being named shows "Unspecified (someone volunteered)".
- **Confidence is per segment, not per word**, because the hosted API does not return word probabilities.
- The sound-alike check cannot see a dropped word when the rest still sounds similar ("move to launch" → "the launch" scores 0.60). The prompt forbids it; the threshold should be tuned on more recordings.
- Hosted model names change; check them before running.